

Domain-Driven Design applied to land administration system development: Lessons from the Netherlands

Peter Oukes^{a,*}, Marc van Andel^a, Erwin Folmer^a, Rohan Bennett^{a,b}, Christiaan Lemmen^{a,c,1}

^a Cadastre, Land Registry and Mapping Agency Kadaster International, P.O. Box 9046, 7300 GH Apeldoorn, the Netherlands

^b Swinburne Business School, Swinburne University of Technology, Hawthorn, Australia

^c University of Twente, Faculty of Geo-Information Science and Earth Observation/ITC, P.O. Box 217, 7500 AE Enschede, the Netherlands

ARTICLE INFO

Keywords:

Domain-Driven Design
DDD
Event-based modelling
Implementation
Interoperability
Land registration
Land administration
Land administration domain model
LADM

ABSTRACT

The introduction or renewal of information systems conventionally begins with data modelling. In the domain of land administration, like others, the process is challenging: complex laws and regulations, lengthy process descriptions, shared organisational responsibilities, differing information encodings and formats, and seeking compliance with the LADM ISO 19152 standard, must be considered. Between 2016 and 2018, The Netherlands' Cadastre, Land Registry and Mapping Agency – in short, Kadaster – successfully undertook the renewal of the information system supporting its deeds registration. The previous system dated back to the 1980s. In-house data modelling specialists led the program, the most extensive undertaken in decades. Inspired by action-research principles, the process and resultant lessons are documented using a case study approach. It is shown that beyond Model Driven Architectures, other model-driven methodologies, such as Domain-Driven Design, are entirely useful in the land administration domain. A domain is usually more extensive than a few objects, and to make it more manageable, DDD divides a domain into subdomains. The DDD term 'problem domain' is used to define a functional area within a context such as an organisation or department. The terms domain in DDD and LADM have common characteristics as considering contexts such as a land registry and a cadastre. Evans (2003) and Vernon (2013) articulate how DDD is a set of design practices, techniques and fundamental principles, terms, and implications to facilitate the development of software projects within complex domains, used to guide software developers and domain experts to share and represent models of knowledge from the domain. In DDD, the ubiquitous language is also essential in intersecting the jargons between domain experts and IT experts. The LADM provides a formal language for describing similarities and differences for describing the many aspects of the land administration domain. The approach demands greater participation from domain experts: they lead modelling of the current state and its evolution: the events. The Annex N of ISO 19152 describes that LADM covers both event-based and state-based modelling via LA_Source and VersionedObject. The application of Event-Based Modelling and Event Sourcing is still relatively novel to the LA domain. Event Sourcing ensures that all current state changes are stored as a sequence of events, enabling querying and state reconstruction. The new information system is considered futureproof, delivering improvements for deed registration times, monitoring, traceability/auditing, history management and interoperability. Further research suggestions include undertaken Domain-Driven Design in other contexts, particularly those implementing LADM.

1. Introduction

There are many issues to consider when introducing or renewing a land information system to support land administration activities and processes. Whilst much focus is often placed on information or data

model design, also demanding consideration are laws and regulations, processes descriptions, responsibilities shared between multiple organisations, and the exchange of information in many different formats and encodings (Oukes et al., 2019).

Another more recent design consideration is the desired level of

* Corresponding author.

E-mail addresses: peter.oukes@kadaster.nl (P. Oukes), Marc.vanAndel@kadaster.nl (M. Andel), Erwin.Folmer@kadaster.nl (E. Folmer), c.h.j.lemmen@utwente.nl (C. Lemmen).

¹ www.itc.nl

<https://doi.org/10.1016/j.landusepol.2021.105379>

Received 16 July 2020; Received in revised form 20 February 2021; Accepted 23 February 2021

Available online 5 March 2021

0264-8377/© 2021 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

system interoperability. The Land Administration Domain Model (LADM), ISO standard 19152 (ISO, 2012), contributes to this. Although LADM is not intended to replace existing systems, it requires attention when integrating this standard into the design process, design tools and competences.

To date, LADM has been implemented at the country level via the development of local profiles. Interestingly, these have mostly been developed by academia, or with industry supportive, as demonstrations or for compliance checking of existing data models (c.f. Kalogianni et al., 2019; Janečka et al., 2018; Vučić et al., 2013; Janečka and Souček, 2017; Bydłosz, 2016; Radulović and Sladić, 2017). That is, the agency responsible for the national land administration has not always been engaged. Kalogianni et al. (2019), for example, shows many countries are developing a local profile. Among those involved in technical implementation, there seems to be unfamiliarity and adaptability to Domain-Driven Design (DDD), Event-Based Modelling, Event Sourcing, and an ubiquitous language. Instead, the aim is to provide a formal language for describing land information systems so that their similarities and differences can be better understood (ISO, 2012). Overall, this suggests that despite the growing discourse and shared language relating to land information system design and implementation globally, there remain relatively few robust case descriptions detailing production level practical implementations.

In the context of the Netherlands, since the early 2000s, the country has been a leader in supporting the development of the LADM as an international standard, developing approaches for implementation, and supporting country-level profile development in other country contexts. In this regard, it is interesting to study the country's recent 2016–2018 redevelopment of its land information system that supports its deeds registration system. The agency with the mandate for delivering land registration in the Netherlands, and consequently the supportive information systems, is The Netherlands' Cadastre, Land Registry and Mapping Agency (hereon in 'Kadaster'). Kadaster performs its public tasks in service of society.

To meet society's demands, like other government agencies, Kadaster recognises the need to undergo a form of digital transformation (c.f. Hess et al., 2016). Specifically, it recognises the need to renew its legacy information systems, first developed in the 1980s, to meet current and future societal demands better. On this, an increasing number of complex legal facts were needing to be registered in the obsolete land information system (known internally as AKR/HYP), with increasing difficulty. The old system, developed using Cobol, was technically, economically, and management-wise, at the end of its life cycle (+30 years). At the same time, Kadaster recognised the need to improve both the information system and the underlying business processes.

Therefore, acknowledging - (i) the lack of detailed case descriptions on contemporary land information system development at production level; and (ii) recognising Kadaster's recent information system renewal. This paper presents findings from the Netherlands context to further enrich the global discourse on LADM implementation using Domain Driven Design (DDD). Specifically, the paper seeks to introduce the DDD approach utilised by Kadaster and ascertain lessons for future land information systems based on the LADM. To this end, Section 2 provides background and context on Kadaster and the need to update the information systems supporting its deed registration system, alongside a background on current systems development concepts and approaches. Section 3 describes the case study methodology applied in the research and the DDD approach as it was explicitly used in the context of Kadaster. Section 4 describes the results of the development methodology, as experienced in Kadaster. The section concludes by detailing key lessons and points of attention. Section 5 examines the potential implications of DDD for the land administration sector generally, areas for future research, and future developments for the LADM.

2. Background

2.1. Background to Netherlands systems renewal

The Netherlands uses an 'improved' deeds registration system (a semi-positive system) (Vos and Roes, 2020) to enable land registration nationally (c.f. Henssen, 2010). A deed system is seen as a negative system (Zevenbergen, 2002). Whilst a title registration system, a positive system (Zevenbergen, 2002), used variously in other country contexts, provides absolute proof of ownership, backed by a state agency or authority, a deed system does not. This means the transfer of parcels between parties is recorded in a notarial deed drawn up by the notary. This deed is recorded by Kadaster, and the recording acts as mere evidence that a transaction has taken place, not absolute legal certainty. To enable the system, each land parcel is given a unique Identifier issued by Kadaster, and this must be included in the deed. The deed is then presented to Kadaster and registered in the public register. As mentioned, Apart from the names of the rightsholders (parties), the rights, responsibilities and restrictions associated with spatial units and the spatial units themselves, the transaction details are recorded to demonstrate that the transaction took place. The transacting parties receive a note from Kadaster that the deed has been registered.

In the Netherlands, the cooperation between the notarial profession and the registrar, residing in Kadaster, is substantial in terms of robust processes and transparency. This is reflected in the high levels of trust afforded the information in the registry by citizens, banks and the courts. That said, the land register does not provide absolute proof of ownership. However, with its accurate and trusted administration, the system objectively provides an acceptable level of legal certainty in daily practice: the entire real estate sector in the Netherlands is familiar with the system and uses it. By checking the register in terms of chains of transactions over a specific parcel, the owner can be determined with a very high degree of certainty (Zevenbergen, 2002; Vos and Roes, 2020), the so-called semi-positive system of land registration.

Information systems to support the deeds system were introduced as a complete system in the 1980s (eventually known as AKR/HYP), using development methodologies, technologies, and languages available at that time. By the 2010s, it was determined that Kadaster would introduce a new way of registering land by the automatic processing of documents based on business rules, supported by an appropriate land information system.

Driving the redevelopment, Vos and Roes (2020) argued that in the old information system (i.e. AKR/HYP), mutations were immediately registered, immediately applied, ready to query and were subsequently validated. For auditing and later reference, the system linked the actual deed ID in the transaction. A record was simply deleted from the registration system in case a right *in rem* had ended. On this, historical tracing is obviously essential, and therefore a daily summary of transactions was compiled as part of the old system processes. This allowed historical investigations, if and when they were required. However, Vos and Roes (2020) mentioned that these summaries did not contain all the required information: manual research had to be executed by searching many deeds, a very intensive and time-consuming activity. According to Vos and Roes (2020), another downside of the old system was that a recording error in the system could only be corrected with an extra technical transaction performed, in the same way as a regular legal transaction. In cases of mistakes, the only option was to execute a correcting technical transaction as a deed, creating extensive overhead. It was envisaged that a new registry system could help solve these issues. It was deemed that any new system design would be oriented towards making transaction processes efficient via automation of business rules to verify the transactions against the registry.

The replacement or renewal of a legacy information system is not a new undertaking in itself. Indeed, other country contexts have had such experiences in the contemporary era. However, the application of concepts and tools relating to DDD to the land administration domain,

specifically the further improvement of a deeds registration approach, is considered novel. Before examining the Dutch experience with DDD, a brief contextual background on this approach's emergence is provided.

2.2. Background on Domain Driven Design

DDD emerges from the suit of systems development approaches that rely upon an interaction between modelling activities, domain expertise, platform independence, and systems architecture. These approaches are not new to the land administration discipline. For example, according to Van Oosterom et al. (2013), the LADM is based on a Model Driven Architecture (MDA). MDA is a software design approach for the development of software systems. It was launched by the Object Management Group (OMG) in 2001 (OMG, 2001). Zaragoza et al. (2016) describe how MDA helps to transform one Platform Independent Model (PIM) into several Platform Specific Models (PSMs), each for a platform or technology in which the final system will be deployed. Automatic code generation is preferable to implement the system for those platforms from the corresponding PSMs. According to the Cephas Consulting Corp (2006), tooling is an essential element in an MDA environment. Janečka et al. (2018) illustrate how developers have tried to make their tools more interoperable by using common standards such as the Unified Modelling Language (UML). Meanwhile, Kahani et al. (2018) show 60 transformation tools for Model Driven Development (MDD) and Model Driven Engineering (MDE) is a well-known tool in land administration model transformation. Interestingly, INTERLIS (Germann et al., 2015) is not on the list. INTERLIS is a Swiss standard (Swiss Confederation, 2007) and is a Data Exchange Mechanism for Land-Information-Systems (INTERLIS, 2006), which support the MDA principles (OMG, 2014).

Supporting MDA activities is the domain modelling itself (Evans, 2003). There are numerous resources enabling domain modelling: Domain-Driven Design provides one example. Evans (2003) and Vernon (2013) articulate how DDD is a set of design practices, techniques and fundamental principles, terms, and implications to facilitate the development of software projects within complex domains, used to guide software developers and domain experts to share and represent models of knowledge from the domain. Domain-Driven Design has three core principles: 1. Focus on the core domain and domain logic; 2. Base complex designs on models of the domain; and 3. Continuously collaborate with domain experts.

DDD is primarily about defining and modelling the wish or problem in a domain such as LA and not about the solution itself; this follows later. The DDD term 'problem domain' is used to define a functional area within a context such as an organisation or department. A domain is usually more extensive than a few objects, and to make it more manageable, DDD divides a domain into subdomains, a so-called collection of Bounded Contexts (Fig. 2).

'Bounded Contexts' are where a boundary is created for each of the concepts, with its properties and operations, and has a special meaning. This must be done without reference to the solution itself. That is, there should be independence of technology and software architecture. Bounded Contexts can be expressed in a context map. Subsequently, the subdomains can be modelled in more detail using the Domain-Driven Design rules. This is expressed generally in Fig. 1. Translate this figure into a specific example of the subdomains of Kadaster without being complete in Fig. 2.

The developed domain model acts as a 'Ubiquitous Language' (Evans, 2003) to help the communication between software developers, architects and domain experts. The Ubiquitous Language is cultivated through the intersection of jargons between domain experts and IT experts. Actual system behaviours are described as a collection of commands and events. The intention is described as a collection of entities and secured in schemas and code.

Evans (2003) proposes to express domain models in UML class diagrams that leverage UML Classes and represents a domain object, which in DDD is synonymous with the domain concept. Rademacher et al.

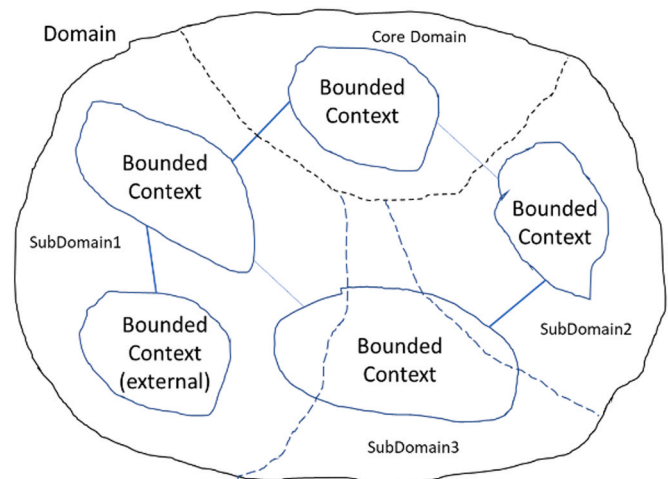


Fig. 1. Example business domain in DDD including subdomains and bounded context after interpretation (Vernon, 2013).

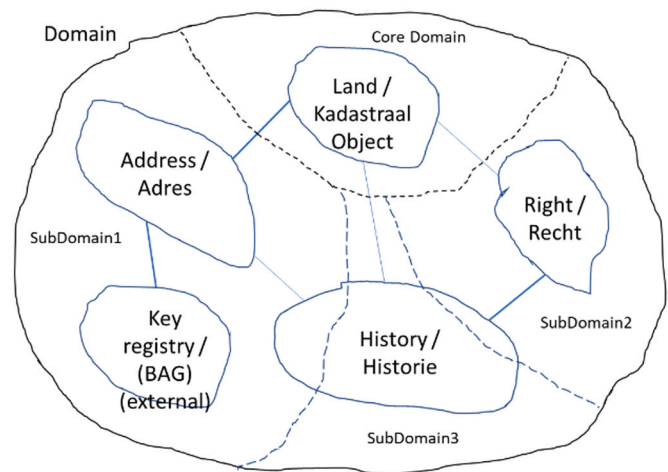


Fig. 2. Example of DDD domains of Kadaster.

(2018) describes a methodology on how to transform a DDD domain model. According to Rademacher, modelling DDD in UML is not formal today: model validation and transformation is not possible yet. The skills and experience of the developer, architect and domain expert are needed.

For actual software development, also a well-known pattern is Model-Driven Engineering (MDE). DDD has many aspects in common with MDE (Evans, 2003). Both use models to represent the domain. As described in the MDE framework, DDD applies the same techniques in practice, such as modelling the domain (structure, rules, dynamics, etc.) and using a domain-specific language (DSL). The latter facilitates the communication between domain experts, architects and developers. Furthermore, MDE complements DDD with model transformation, as described with MDA, and code generation to create the Information system. Having provided a brief overview of the Dutch context and explanation for the DDD approach, the methods for assessing DDD application in the context of Kadaster is now provided.

3. Materials and methods

3.1. Method for conducting research

Action-research principles and case study methods inspired the work underpinning this paper. The research activities were context-bound and

participative (Koshy et al., 2010; Kemmis et al., 2013) and undertaken by practitioners seeking to improve practice via a change process. Likewise, with regards to the case study approach, the work can be understood as an empirical study examining a contemporary phenomenon within its real context (Yin, 2014). This combined research paradigm and approach was considered appropriate for the work: it enabled sketching an overall picture of a phenomenon in its context and later, if needed, enabled generalising to similar cases or lessons.

Alongside Kadaster's development process, data collection for the research case study analysis involved several methods: (i) observations of development processes, (ii) interviews with key-stakeholders; (iii) questionnaires with other stakeholders and (iv) participation within the development team and undertaking consultations - the latter data collection techniques drawing inspiration from action-research methods. In terms of the actual data collection instruments and approaches, each sought to shed light in some way on the following questions: Why did Kadaster choose DDD to replace the existing deed system? How did Kadaster manage to renew the land administration with DDD? What lessons can be learned for future implementations of land administrations based on LADM? (see Annex A for more details on questions). The case questions and analysis were all couched in paragraph four.

3.2. Method for systems development

As explained, for the actual systems development work, DDD was finally the primary approach applied. The result did not commence from a blank canvas. A pre-existing knowledge of systems requirements was captured and fed into the process. Also, the pre-existing logical Information Model Kadaster (IMKAD) (IMKAD, 2019), the information model of land administration in the Netherlands, acted as a basis. IMKAD models all real estate's descriptive and geometric data in the Netherlands and the rights and rightsholders. IMKAD is the information model of the Key Register of Cadastre (BRK). This Key Register consists of the cadastral registration of properties and the rights that have been established, and the cadastral map. This map shows the cadastral parcels, including the parcel numbers and the state's boundaries, provinces and municipalities.

IMKAD is a national standard for geo-information in the Netherlands. IMKAD (IMKAD, 2019) is LADM (ISO, 2012) compliant (Leenders et al., 2014).² Fig. 3 show the domain model in UML.

To further explain the DDD process in more detail, it is necessary to define two architectural patterns are introduced: Command Query Responsibility Segregation (CQRS) and (ES). These were applied to Kadaster's information system development work. Here, CQRS is explained through an example transaction in combination with ES. A key concept is 'events' – these being the result of transactions. The pattern of events is known as ES, and this is subsequently described.

In former days, the transfer of a right on a parcel from party A to party B used paper-based instruments. In the Netherlands, there would be books at the local office where each piece of land was registered. A human-based interpretation of the deed was processed. Against the eligible book, the appropriate page was selected. The transfer of ownership, using the 'strike through' of the parcel related to the former owner and writing-down the parcel associated with the new owner, took place. The paper registration was replaced when computer systems were introduced with a database as structured storage. However, the same processes were implemented, albeit digitally, as 'the paper' based. The data model was as optimal as possible (normalised) to represent the 'current' state only: storage and computing capacity were expensive.

² Additionally, the Netherlands is a member state of the Europe Union: the European standard for the exchange of parcel information is INSPIRE (2014), and it supports parcel information according to LADM (ISO, 2012). IMKAD also complies to INSPIRE as well.

However, the approach meant that not all transaction intelligence histories were tracked. In contemporary times, more data than ever is processed and stored, made possible with the help of the relatively cheap compute capacity and storage.

A deed expresses for something to be done: Stopford (2018) and Vernon (2013) describe it as a 'command' that requests a system state change. Once a deed is accepted, it triggers a process at the Land Administration Office, previously completed by the human or hybrid digital approach – and in information system terms; this requires modelling. The first step is to check the deed compliance. And whether the notary did their job well in respect to the law and regulations. Next, one of the transacting parties, Party A, is checked as to whether they are the owner of the land and can sell it to someone else. If not, the land cannot be sold. Once the deed is validated and accepted, the current state can be edited by 'strike through' the parcel relation to the current owner and writing down the parcel relation to the new owner. A stack of 'Events' can be built by adding an ID for each accepted 'command' and a time-stamp of acceptance. These 'Events', facts and notifications, should happen automatically upon registration and expect no action and no response (Stopford, 2018).

Events are expressed in the past tense. Whilst the 'command' is a deed saying 'transfer ownership', the 'event' expresses what has been accepted and changed in the registration, i.e. 'ownership transferred from Person A to Person B'. The view on who is the current owner of the parcel comes from the stack of events. These are queries taken from a repository of the events, itself a result of the commands. Queries enable investigation of the repository without changing the registration state (Stopford, 2018; Vernon, 2013).

Fig. 4 provides a graphical representation of the entire process – and this is the outcome from the architectural patterns 'CQRS' process and is considered fit-for-multi-user collaborative applications (Dahan, 2009). According to Dahan (2009), the characteristics of CQRS are for creating more straightforward and more scalable constructs. By understanding the users intent and the process in the head of those users, it is possible to model the user interface. A maintainable system with CQRS requires understanding the business requirements in detail (Dahan, 2009). Zevenbergen et al. (2007) describe the dynamic aspects of modelling concerning domain models and use cases. It is essential to model the commands from the user interface, the user interface itself, the events from the process(es) and the domain model. In his case study, Fig. 5 shows a translation of the model into a component as designed by Kadaster.

Taking the concept of 'events' further, it is essential to note that things happen all the time – sometimes simultaneously, sometimes in parallel, and also sequentially. Computer systems are similar to our physical world in the way that things happen all the time, i.e. Event-Driven patterns. Therefore, the information system needs a data model that describes all the changes in the registration. This is called Event Sourcing (ES) and is in line with Annex N of the LADM (ISO, 2012) on event-based modelling.

To unpack this concept, continuing with the abovementioned transaction example, a change in the registration occurs after the transaction is verified and valid. This produces an explicit immutable audit trail of how and why the registration has been changed at a specific point in time. In this case study, Kadaster modelling experts showing they designed an event such as EventAanleiding (EventReason) with an event identification object Compositelid as shown in Fig. 6. The history on a specific parcel, owner, or right is in this way also open: a query of the chain of events in the Event Store.

An Event Store is, in practice, a file on a file system in which each line represents an event. Other systems can access the events by (i) subscription (i.e. every new event on a specific characteristic will be sent to the subscriber, which triggers a process), or (ii) by creating an info-store such as a database. In addition, all statistical or trend analysis and time travel are also possible. This can be done independently from the core system: the information is copied from the Event Store with no impact

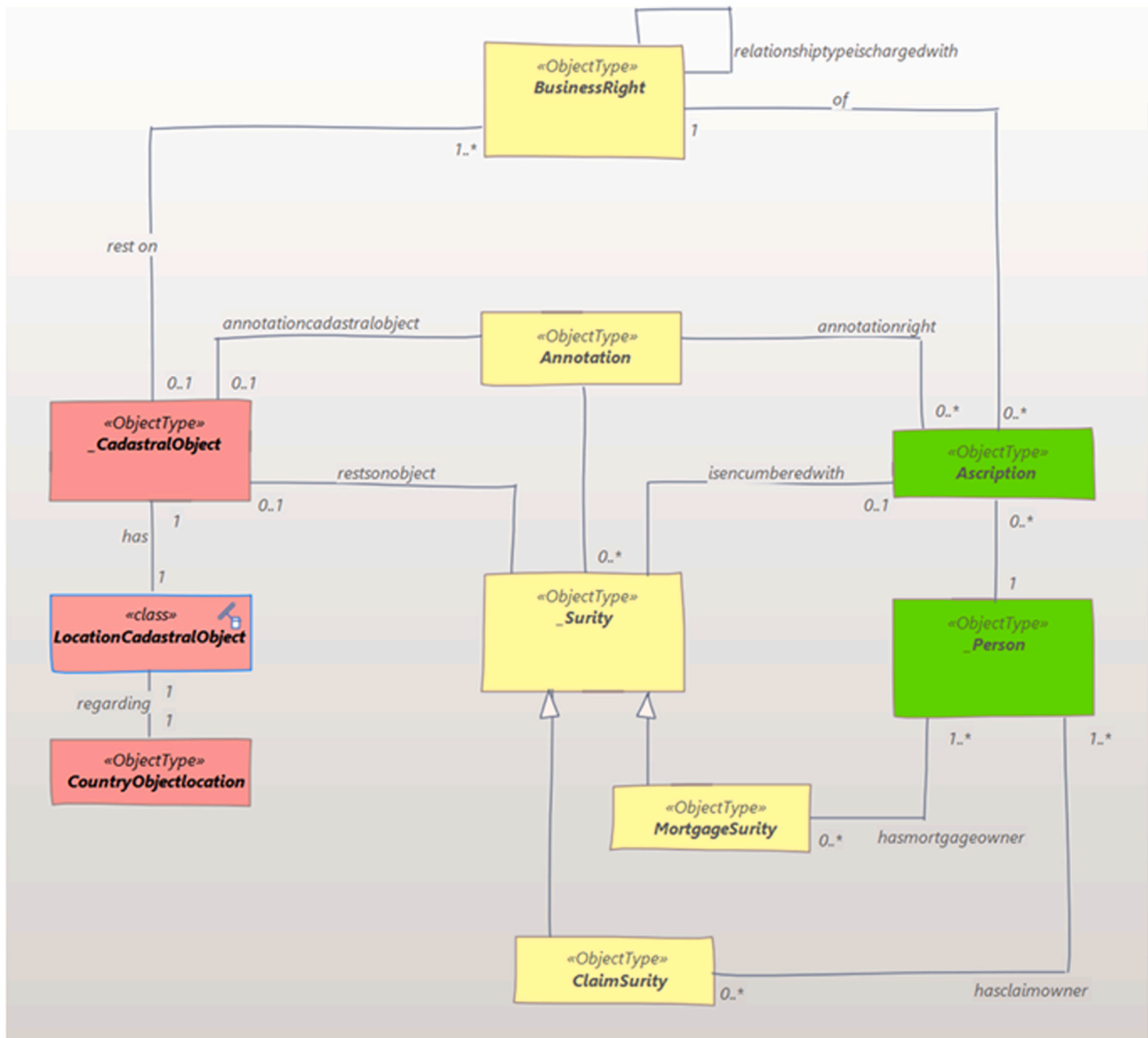


Fig. 3. Domain model in a UML model of the deed registration system of Kadaster (IMKAD, 2019) using the LADM package colours. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

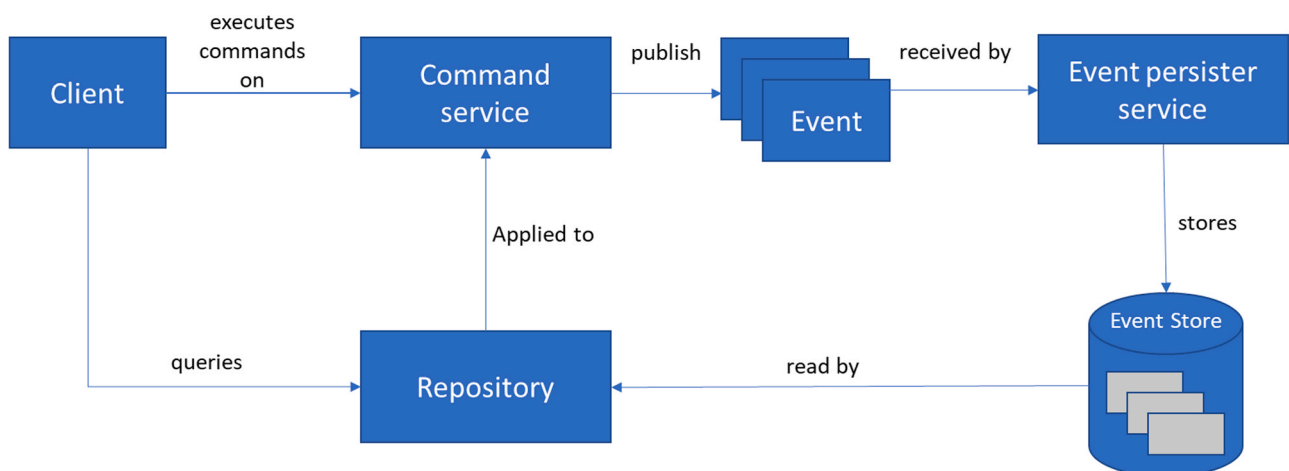


Fig. 4. Visual representation commands, queries and events after (Vernon, 2013).

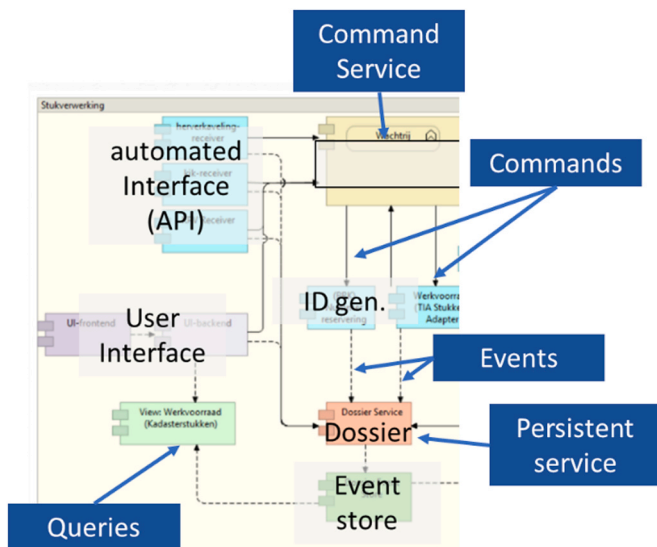


Fig. 5. Implementation model of components: commands, queries and events.

attribuut
Compositelid aggregatelid
LogischTijdstip logischTijdstip
TechnischTijdstip technischTijdstip
String user
Stuklid onderhandenStuk
StukdeelId onderhandenStukdeel
Stuklid formeelStuk
StukdeelId formeelStukdeel
Stuklid verbeteringStuk
<u>StukdeelId verbeteringStukdeel</u>

Fig. 6. Example of attributes of an event designed by Kadaster.

on the core processes. Fig. 7 provides a simple overview.

With respect to a relational database theory and practice, the record involved is updated in the database after validation of the state change. Without additional measures, the previous state is not available anymore, and also, who made the change and when the change took place is unknown. This is in line with LADM (ISO, 2012) state modelling – in Annex N of the standard. With versioned objects in a state-based system, an event-based modelling approach can be designed where an instance of LA.Source represents the events.

A final aspect worthy of mention is ‘separation of concern’ in information processing and information publishing. In general, relational databases should be tuned to support transactions and other business processes, such as publishing and analytics. Usually, the separation of transactional and analytical database environments are realised (Man-nino, 2012; Bennett et al., 2019). Additional measures are needed to keep the processing database, the publication database in sync and the

transactions history. The concept of event sourcing has some advantages that must be specifically designed and configured in a relational database.

To support ‘separation of concern’, all changed records are stored in the Event Store. Events are immutable, and the Event Store is append-only. All the Events stored forms establish the history too. Even the corrections by employees and the registrar are noticed as events and cannot be removed from the Event Store. There is never direct access from the client to any Event Store. Only authorised services have registered access. Concerning LADM (ISO, 2012), the standard supports both state and event-based modelling. For publishing and analytics, there are Info Stores created from the Event Store. The Info stores can be customised for the purpose of the process or customer needs, such as partial data or only actual data. The Info Stores can be created and synchronised at any time by a synchronisation stream, and only authorised services have registered access.

With the overarching research methodology and the parallel systems development methodology (i.e. DDD) now explained and justified, results of those applied methods are provided in the subsequent section.

4. Case results and discussion

With the renewal of the land information system supporting deed registration in the Netherlands, the goal of Kadaster was to introduce a new way of registration with an embedded ES philosophy. Therefore, the focus would be on transactions, reflecting this core aspect of deeds systems, and on automated business rules to verify the transactions against the registration.

The scope of the new deed registration information system, KOERS, the ‘Cadastral Objects and Rights Registration System’, required that the old system’s functionality (registration of legal facts) was also reflected in the new system. For many years, business experts worked on a clear and more legal based list of transactions that might be executed on the registration to update it. This evolved list was taken as a base for the new system, including the draft version of IMKAD (2019).

With DDD, the notion of movement of the data in the model was a clear concept closely related to the concept of these legal facts that could happen in the registration. Therefore, during development, Kadaster explored and experimented with the legal fact, the aggregate boundaries and the events to understand the domain, to model actions (commands) in the form of legal facts and implications of these into events changing the state of the model. Additionally, functionality such as automated processing of deeds, correct parcel support (with provisional and administrative limits) needed to be included to meet the requirements for basic registrations. This also necessarily included capturing history, referral via identifying characteristics, and facilitating feedback and signalling ‘in research’.

The choice for DDD only followed after initial work was completed with ES. The most important function which the new system had to support was the audit trail and history model, which ES enabled. The choice for CQRS and DDD followed more or less naturally from the perspective of software developers. DDD places concepts as CQRS and ES in a certain context. ‘Separation of concern and responsibility’ support the non-functional need for decoupling: every bounded context has to function on its own. In the meantime, sharing information is essential too: loose coupling is a better approach. The next sections describe how Kadaster managed the modelling of the new deed registration information system and which tooling was applied.

4.1. Modelling process

After using ES, Kadaster needed to find a way to model the new system using a DDD approach - and MDA was applied. The Model-Driven Architecture (MDA) approach is a sequential design process. DDD is a design approach in an agile concept and fits the state of the art of a cloud-based approach and microservices. This is novel to land

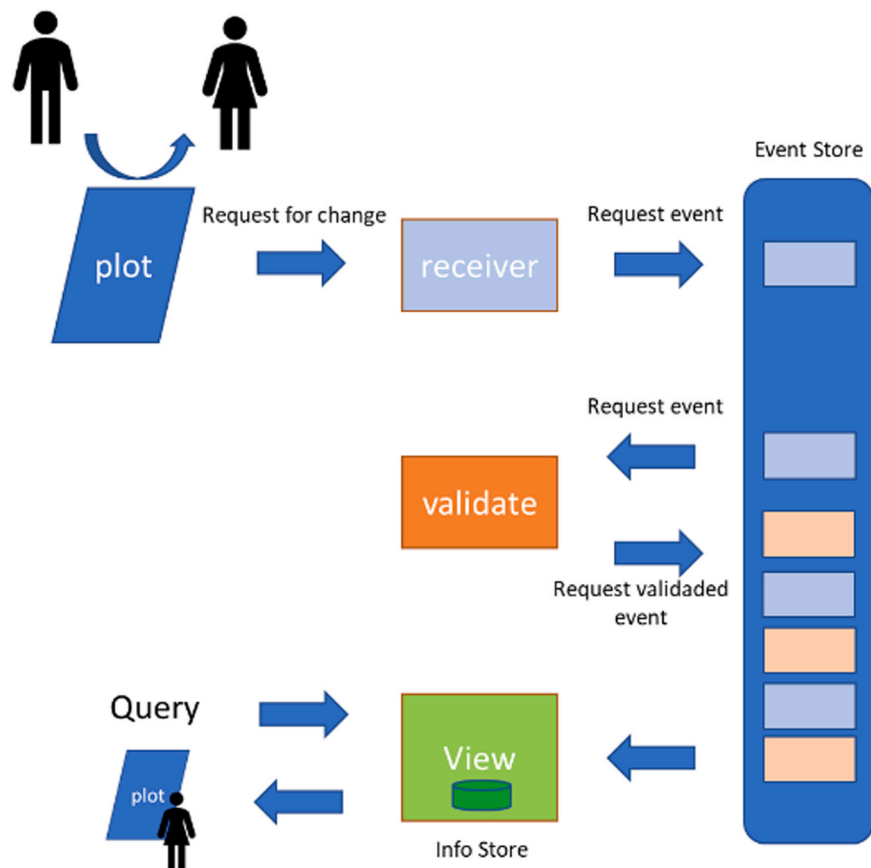


Fig. 7. Simple overview of CQRS/ES services implementation inspired by [Stopford \(2018\)](#).

administration design and implementation. As [Vos and Roes \(2020\)](#) stated, modelling specialists needed to identify new modelling techniques for CQRS. This required separating ‘Commands’ from ‘Queries’. What is an order (command) to change the registry state, and what is this order’s intention? Or, what are ‘simply’ just questions (query) that need to be answered? According to [Vos and Roes \(2020\)](#), this Event-Driven approach is a paradigm shift in the registration, which equally applies to the case of Kadaster. In short, the Event-Driven approach provides concepts to align business experts with software developers. Instead of complex models and business rules with only business jargon, developers are promoted to understand these models and rules. Thus, business experts understand the developers’ exact and straightforward way of thinking, clearing and optimising the business models and rules.

The first step in the modelling process involved translating the logical data model IMKAD ([IMKAD, 2019](#)) into a DDD domain model. [IMKAD \(2019\)](#) was in the draft version available during development. According to the IT experts, the potential conflicts and impacts of translating DDD into the IMKAD were uncertain. The modelling of the events was absent in the IMKAD model and are in [IMKAD \(2019\)](#) with more detail but still on a high level. IMKAD models all classes with relationships; however, there is an explicit focus on aggregate and context boundaries with DDD. DDD primarily builds from a domain model divided into several subdomains (bounding contexts) and aggregate roots. The ‘art’ is to identify an appropriate grouping. Based on the IMKAD domain model Kadaster identified the following aggregate roots: (Cadastral) Object, Subject, Deed (Source Document), Location and Processing Case. The new KOERS information system supporting the processing of the deeds contains the Processing Case aggregate in its bounded context, and the (Cadastral) Object, including all rights *in rem* in the registration bounded context. It also contained references to external bounded contexts for Subjects, Deeds and Location. These

contexts were separated for several reasons, one of them being the privacy regulations of the EU.

According to the modelling experts and developers, grouping should be carefully chosen, ensuring delineation consistency. For example, the IMKAD model was considered not to have been entirely converted for the registration component: only those entities and attributes required for deed validation and event creation were incorporated.

To create models for CQRS and ES from data models was difficult because the logical data model, IMKAD, tried to solve all functionality for registration and provision/publication in one single model. In retrospect, as [Bittner and Spence \(2003\)](#) described, it was essential to develop the Use Case model and Use Case specifications, from which the commands could be derived and the business rules and validation rules. In summary, this is an example of vertical interoperability (i.e. translation of a business activity to a process level).

4.2. Modelling tools

Generally, automatic code generation is preferable to implement the system to the corresponding Platform Specific Models – and as discussed, in theory, the software can be generated. However, in more traditional software development specifications and business models, automation is often not possible as it produces suboptimal code. This was certainly the experience in Kadaster: the logical model IMKAD had to be converted manually, almost one on one, into Domain Model Java classes, distinguishing between Entities and Value Objects (immutable data types) according to the DDD approach. According to the questionnaire IT developers say, it was easy to manually convert the classes that implement entities and data types: relatively speaking, the model was not large. Moreover, commands and events were not modelled in the logical model IMKAD: an automatically generated implementation would be incomplete. Additionally, at the beginning of the system’s

development, it was not clear what the final implementation would look like: according to the modelling experts of Kadaster, the final view was discovered along the way.

4.3. Implementation

Once the new information system supporting deed registration was deemed mature enough to start using, the system went into a semi-production environment. According to the IT experts, the idea was that the actual production was in the old system. However, every transaction request for change would be mirrored in the new system. In this way, there was an understanding of the theoretical and designed system before full production, and checks between the new and old system were possible. According to the IT experts, the adjacent systems, which exchange information, also improved the new system.

The parallel system approach was enabled for a whole year. Once proved the new system's quality was equivalent to the old system and the functional scope and improvements realised, the next step involved going live. This operation was a 'big bang' process completed over one weekend. The old system was stopped, and transactions underway were finished. The state of the old system was migrated into the new system as events in the Event Store. After a check on the state of both systems being identical, the new system was started, and all (updated) adjacent systems pointed to the new system. After a test run, the IT experts checking that the system processes were functioning with expected outcomes once again, the system was live. In principle, there was no way back to the old system. After 1 day of production with the new system, there was a point of no return.

Even after a year of parallel operation, in practice, many issues arose. The business and IT employees worked together to resolve the problems within a limited time as possible. Examples of issues were known unsupported and legal rights that are rare being presented earlier in the system's lifecycle as expected, such as stacking more than three rights *in rem* on a plot or changes in the law or justified wishes of users sometimes required functional adaptations. By applying DevOps, Agile and Continuous Delivery techniques, these 'new' types of deeds were realised by the developers and business together. The system was extended with this functionality within a few days. Another issue was the slow performance of the building of information stores (stored views). The reason was the amount of data had to be filtered out of the Event Store. In the dry run, the Event Store was not as large as the real production

Event Store, after transforming transactions from the old system into the events for the new system.

4.4. Integration

Linking with other government key registers (e.g. persons (BRP), business (NHR)) and systems can be done in two ways: (i) with an info-store, and (ii) with a subscription to events through a web service. The info-store is modelled explicitly according to the customer's information (events) needs and made accessible by a web service. The info-store is created at the startup of the service and filled with events from the Event Store of a certain topic. New events are added over time. With an event subscription, a customer can subscribe to the flow of events and process and apply them internally at their own discretion (Fig. 8). The implementation model of service components in the combination of event and info stores is shown in Fig. 9.

A point of attention are events in which encodings are used to exchange information. A context map describes which data is exchanged and which encodings are applied between bounded contexts for all relationships in the domain model. Kadaster has mainly used Representational state transfer (REST) web services, with encoding (Geo) JSON (JavaScript Object Notation), GML (Geography Markup Language) and XML (Extensible Markup Language). For the external systems, the encoding was mainly XML which comply with the logical model IMKAD. In the opposite direction, when data from an adjacent system is received, it must be delivered following the policies of IMKAD. This is so that it fits with the model of the system. In DDD, a so-called anti-corruption layer will have to be designed for each link to check and transform the data and encoding when needed. According to Evans (2003), an anti-corruption layer is a way of linking two Bounded Contexts. When there is another system to interact with, there is no complete understanding of the model used, and there is no control over that system. The anti-corruption layer is a separate layer that can reduce the friction between two systems when there is a need for functional integration (i.e. when a legacy system is involved). This is an example of DDD helping to support horizontal interoperability between systems.

4.5. Interoperability

DDD contributed to interoperability. This was assessed regarding the CEN/ISO standard 11354-1:2011 (ISO, 2011). The relationship between

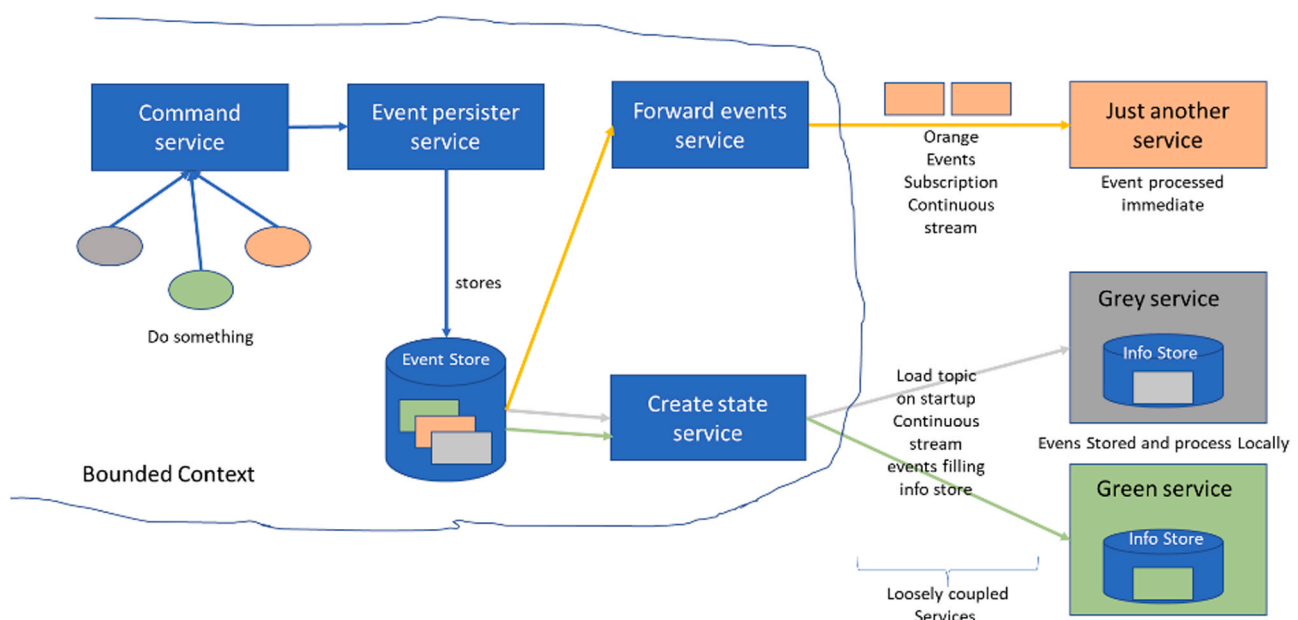


Fig. 8. How to use events to publish by an Info Store or trigger other systems by the continuous stream of events.

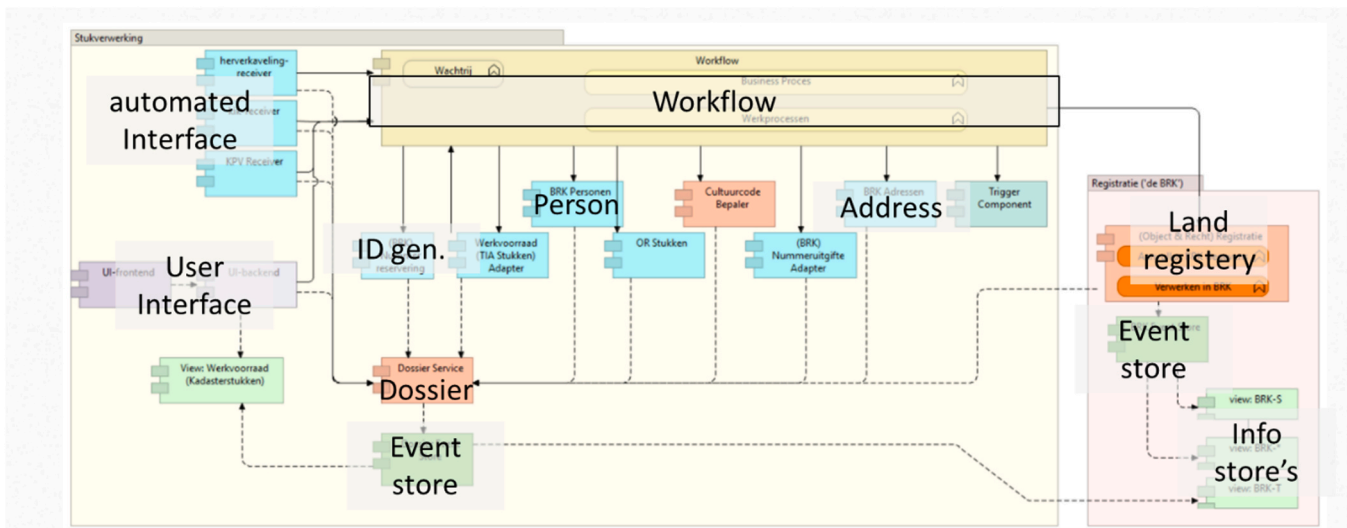


Fig. 9. The implementation model of services and (event and info stores) of Kadaster.

Business, Process, Service and Data was fully managed by the DDD design process, mainly through the common language developed between domain experts, architects and developers. As described by Oukes et al. (2019), the barriers, including the incompatibility between entities within the enterprise, obstructing the exchange of information and other entities, were covered by the domain model approach. As described by Chen (2009), Typical Barriers, including different semantics and syntaxes used in other process modelling languages, incompatible process execution engines and platforms, were all found to be mitigated by CQRS and ES. Further, the concept of decoupling and subscription to events, and the use of Event Stores, created a secure supply of the needed information of the adjacent systems.

4.6. History

On documenting history, the advantage of ES is that the implementation of the required two-dimensional history was considered easier than alternative strategies, such as Bitemporal Modelling (which was the historical approach used in Kadaster). In LADM (ISO, 2012), a standard Class is a Versioned Object, introduced to manage and maintain historical data in the database. History requires that inserted and superseded data are given a time-stamp. In this way, the contents of the database can be reconstructed, as they were at any historical moment. DDD, in combination with CQRS and ES, enabled the capture and management of this history. In an event-based modelling approach, the event is represented by an instance of LA_Sources. Each change is shown as an event and is therefore included in the Event Store with a time-stamp of creating the event. In this way, the history of changes around an object can be retrieved from a chain of events in the Event Store. After all, the change will result in an event, and a new state as the LADM standard expects.

4.7. Audit trail

An audit trail is an administrative undertaking through which one can find out, within an organisation, how and by whom a mutation or transaction has come about. An audit trail can be used to check whether a transaction has been authorised or if there has been a fraud, for example. As a key societal register, it is important to have a land registration system that can be trusted, meets the highest security requirements, and can be guaranteed by the registrar. ES enables robust audit trail capture in the new system: events (i.e. state changes) are a core element of the system. They are stored, immutably, in the order, they were created. The resulting event log provides a comprehensive

audit of precisely what the system did, by whom and when, as in the versioned object with CI_ResponsibilityParty of LADM. In cases of corruption, it should be possible to re-derive the system's former state by rewinding the event log and replaying the events in order. Also, administrative additions can be covered too. These are new events generated on behalf of the registrar to support corrections, for example.

4.8. Events driven

The disconnect between processing (commands) and publishing (events), enabled by CQRS and ES, improves the communication between different services. The experience was that info-stores contain much historical data too, and not all of that is always necessary. When building the Event Store, this "unnecessary" data takes much time to compile. Therefore, it is important to understand which data consumes demand. Failure to properly model business events is often the biggest problem in a decomposition process using CQRS and ES. Modelling the right events is an iterative process. Once clear which data is needed, the info-store can be created from the Event Store. In a CQRS/ES environment, there is no longer just one "the" database. Sometimes for, i.e. analytics, an ad-hoc query on an info-store is complicated if not all attributes are included. In this case, an additional relational database can be created from the Event Store for analytics. The registration system's advantage is that it does not degrade the performance because of running the analytics scripts. At any rate, this generally aligns with contemporary approaches to the separation of transactional and analytical database environments (Mannino, 2012; Bennett et al., 2019).

5. Experiences and lessons

Post-2018 implementation, after a year of continuous operation, several lessons were evident. First, fundamental to the DDD methodology is defining, sharing, and discussing the domain model, including subdomains and bounded contexts, and connect the domain experts, analysts, and architects. A combination of top-down and bottom-up expertise was found to work well here. This fits with the agile working concept of Kadaster: an infinity loop of design, develop, test and improvement. Creating the common (and ubiquitous) language is part of this, so every team member knows and understands the domain. Commands, Aggregates and Events are explored and are respected by the business analysts (top) and the developers (bottom). Both the business experts and developers applied the ES way of thinking promoted within DDD. This is on one hand a form of implementation of the system, but, on the other hand, it is a way of expressing business model evolution and

therefore business rules - clearly and concisely for the business experts and developers. During development, this became even more clear because business experts could verify the team's automated tests very quickly and confirm the new system behaved as functionally intended, including the flexibility of implementing new laws and regulations.

Second, it is important to connect events to process events in the domain during the design process, aligned with the actions performed. Most of the time, this interaction is not included in the logical model. Events should be seen more as functional building blocks. According to domain and modelling experts, the knowledge about using them should be shared in the project team. In a logical information model, usually, no user-interface is modelled, yet it is an entry point for a process with interaction. Therefore, a point of attention is to create models for UI and commands from the process that use the UI.

Third, concerning system maintenance, this is considered a positive experience for Kadaster.

Undertaking development in-house may account for some of this positive result. After all, the team is both administrator and developer, meaning bottlenecks and issues are more quickly reported and resolved. The experience is that the system has become more mature, if not complex (as opposed to more complicated), in that the events and, more specific, the creation of info stores contributes to this. Some research has to be done to simplify this.

Fourth, after the success of applying DDD to redesign the deeds registry information system, some new systems have been (re)designed according to the concept. The experiences of the initial DDD project could be stylised. If an organisation opts for DDD with details in CQRS/ES, it is also good practice to fully apply microservices, an emerging trend in IT and the cloud. The agile approach's advantage in continuous Delivery and DevOps is to start small and expand and improve over time. It means there is a working system from the beginning, which is important for the stakeholder to look and feel the system 'to be'.

Fifth, interestingly, the entire registration system was realised with open-source software such as Linux, PostgreSQL, Angular, Scala, Axon Server, Java including Bean Validation and VAVR, SpringBoot and Kubernetes as a framework for microservices. Of course, this requires having the in-house expertise to use and maintain the system developed from these tools.

6. Conclusions and future directions

Moving from system concept to implementation is not a straight forward pathway. This case study of DDD, applied in Kadaster, shows the possibilities of the approach used to the land administration domain more generally. It shows it is possible to approach the development of land registration differently. The lessons help to overcome the complexity of implementation by introducing a more technical approach only on the subject of concern with business involvement without to be technical. By choosing subdomains and concentrating on interoperability concerns to the subdomains, the implementation scope is smaller and less complex. On business and process levels, there is a problem to solve issues between these subdomains.

The development strategy for the new system focused on understanding transactions as events, more closely reflecting the deeds registration approach to record the chain of transactions over a parcel (Zevenbergen, 2002; Vos and Roes, 2020). The Event-Driven mindset offers the opportunity to move beyond state-focused systems, enabling better land registry traceability, functionality and system interoperability. It provides the opportunity to undertake a scaled production-level of LADM implementations. The event-driven approach seems particularly well aligned with deeds registration systems, but there is an obvious utility in title-based systems.

On a more technical level, the Kadaster case confirms that data exchange is made easier with standard formats such as XML, JSON, and GML. Moreover, through Events Sourcing, the interoperability and operationalisation improve by loosely coupled systems, with their own

models, software and platforms. The Land Administration Domain Model is a descriptive standard (ISO, 2012). There are many implementations of LADM described, but there is still a need for examples of encodings and exchange formats that fit. In the development of LADM II, there is a chapter foreseen which will cover this. The lessons of Kadaster can contribute to this.

In terms of the development process itself, Kadaster started experimenting with ES and soon moved from a developer perspective to the DDD methodology. DDD comes with its own set of design practices, techniques and fundamental principles, and terms that facilitate software projects finding a common language for complex domains. DDD impacts when an existing base model exists, mainly the designing of behaviour process and events. DDD guides modelling specialists, software developers and domain experts to share and represent ideas and constructs with models. The design process of DDD provides a solution for interoperability, as LADM proposed because the entire environment of the system is considered. DDD fully appreciates the concern (Chen, 2009) of interoperability between Business, Process, Service and Data, and this seems to have been largely confirmed in the Kadaster case. It was easier to start small and modular with a single part, a bounding box of the domain model, expand the modelling via iterations, and code over time. However, it is essential to recognise that a prerequisite for DDDs application is to have good knowledge and human skills in both the IT and business parts of an organisation (if they are separate).

Positives aside, there were risks of DDD becoming overly complicated – and the potential failure in the modelling business, often an issue using CQRS and ES, was recognised. It is also important to note that contemporary data modelling tools do not fully support DDD, CQRS and ES. While Evans (2003) proposes to express domain models in UML class diagrams, the Kadaster case demonstrated that it is not a trivial issue: DDD syntax is not UML aligned yet. Whilst Rademacher et al. (2018) does describe a methodology for how to map DDD model patterns to UML 2.5 metamodel equivalents, and it is clear that more research is needed for modelling DDD in UML. On conversion from models to code, Zaragoza et al. (2016) and Rademacher et al. (2018) both state that translating remains complicated: it needs human skills and existing models are inadequate for capturing behaviours. Another important obstacle is sharing and exchanging data: Kadaster opted for XML and JSON encoding standards. However, uniform acceptance of these standards is not assured.

Taking the abovementioned challenges into account, it is suggested that further research be undertaken to determine the extent to which DDD can be further supporting in implementing LADM at national levels. Many DDD-related questions require examination, including: (i) What are the optimal subdomains and aggregate roots? (ii) Are the subdomains the LADM packages, or is it more subtle? (iii) Where can the cut be made if the responsibilities are shared over multiple parties/organisations? Here, one suggestion could be to develop a modelling methodology example in the upcoming LADM revision, illustrating LADM implementation using DDD. This could also introduce encoding examples, showing which encodings fit best between LADM packages. Research is needed on how DDD can support both a deed system and title system with LADM as a base model.

CRedit authorship contribution statement

Peter Oukes: Writing - original draft, Visualization, Conceptualization, Validation. **Marc van Andel:** Writing - original draft, Resources, Validation. **Erwin Folmer:** Writing - review & editing, Validation. **Rohan Bennett:** Writing - review & editing, Methodology. **Christiaan Lemmen:** Writing - review & editing, Methodology, Supervision.

References

- Bennett, R.M., Pickering, M., Sargent, J., 2019. Transformations, transitions, or tall tales? A global review of the uptake and impact of NoSQL, blockchain, and big data analytics on the land administration sector. *Land Use Policy* 83, 435–448.
- Bittner, Spence, 2003. *Use Case Modelling*. Addison-Wesley Professional.
- Bydłosz, J., 2016. Developing the Polish Cadastral Model towards a 3D Cadastre. In: 5th International Workshop on 3D Cadastres, 2016, Athens, pp. 505–518.
- Cephas Consulting Corp, 2006. *The Fast Guide to Model Driven Architecture, The Basics of Model Driven Architecture (MDA)*.
- Chen, D., 2009. *Framework for Enterprise Interoperability*. Université Bordeaux, Bordeaux.
- Dahan, U., 2009, December 9th. Clarified CQRS. Retrieved from The Software SimplistEnterprise Development Expert & SOA Specialist: (<http://udidahan.com/2009/12/09/clarified-cqrs/>).
- Evans, E., 2003. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Pearson Education (Us).
- Germann, M., Kaufmann, J., Steudler, D., Lemmen, C.H.J., Van Oosterom, P.J.M., de Zeeuw, K., 2015. The LADM based on INTERLIS. In: *From the Wisdom of the Ages to the Challenges of the Modern World: Proceedings of FIG Working Week, Sofia, Bulgaria, 17–21 May 2015*, 10 p. ISBN 978-87-92853-35-6.
- Henssen, J., 2010. *Land Registration and Cadastre Systems: Principles and Related Issues*. Technische Universität München.
- Hess, T., Matt, C., Benlian, A., Wiesböck, F., 2016. Options for formulating a digital transformation strategy. *MIS Q. Executive* 15 (2).
- IMKAD, 2019. (<https://developer.kadaster.nl/schemas/imkad/20190701/uml/index.htm>).
- INSPIRE Thematic Working Group Cadastral Parcels, 2014. *Infrastructure for Spatial Information in Europe D2.8.1.6 Data Specification on Cadastral Parcels – Technical Guidelines*. European Commission Joint Research Centre, Brussels.
- INTERLIS, 2006. *INTERLIS Version 2 - Reference Manual - Edition 2006-04-13 (english)*. (https://www.interlis.ch/download/interlis2/ili2-refman_2006-04-13_e.pdf). (Accessed 30 September 2020).
- ISO, 2011. *ISO 11354-1:2011 Advanced Automation Technologies and Their Applications - Requirements for Establishing Manufacturing Enterprise Process Interoperability – Part 1: Framework for Enterprise Interoperability*. ISO/TC 184/SC 5 Interoperability.
- ISO, 2012. *ISO 19152:2012. Geographic Information – Land Administration Domain Model (LADM)*. International Organization for Standardization: ISO, Geneva, Switzerland.
- Janečka, K., Souček, P., 2017. A country profile of the Czech Republic based on an LADM for the development of a 3D cadastre. *Int. J. Geo Inf.* 6 (5), 143.
- Janečka, K., Bydłosz, J., Radulović, A., Vučić, N., Sladić, D., Miro, 2018. Lessons learned from the Creation of the LADM based Country Profiles. In: 7th International FIG Workshop on the Land Administration Domain Model, Zagreb, Croatia, 11–13 April 2018.
- Kahani, N., Bagherzadeh, M., Cordy, J.R., Dingel, J., Varró, D., 2018. *Survey and Classification of Model Transformation Tools - Software and Systems Modeling*.
- Kalogianni, E., Kalantari, M.E., Dimopoulou, Van Oosterom, P., 2019. LADM2019. 8th Land Administration Domain Workshop, 1–3 October 2019. LADM country profiles development: aspects to be reflected and considered. Kuala Lumpur, Malaysia: International Federation of Surveyors, Copenhagen, Denmark. ISBN 978-87-92853-92-9.
- Kemmis, S., McTaggart, R., Nixon, R., 2013. *The Action Research Planner: Doing Critical Participatory Action Research*. Springer Science & Business Media.
- Leenders, Jans, Lemmen, 2014. *NL Country Profile*. LADM-NL. Internal Report. Kadaster, Apeldoorn, Netherlands.
- Koshy, E., Koshy, V., Waterman, H., 2010. *Action Research in Healthcare*. SAGE Publications Ltd, London.
- Mannino, M.V., 2012. *Database Design, Application Development, and Administration*. McGraw-Hill Irwin.
- OMG, (2001). *Model Driven Architecture (MDA)*. Milford, MA 01757 USA: The Object Management Group (OMG). Retrieved June 8, 2020, from <https://www.omg.org/cgi-bin/doc?ormsc/01-07-01.pdf>.
- OMG, 2014. *Object Management Group. Model Driven Architecture (MDA) MDA Guide Rev. 2.0*. 2014. (<http://www.omg.org/cgi-bin/doc?ormsc/14-06-01.pdf>). (Accessed 30 September 2020).
- Oukes, P., Lemmen, C., Folmer, E., 2019. Interoperability issues related to LADM profiled implementations – a first exploration. In: 8th International FIG workshop on the Land Administration Domain Model, Kuala Lumpur, Malaysia, 1–3 October 2019.
- Rademacher, F., Sachweh, S., Zündorf, A., 2018. *Towards a UML Profile for Domain-driven Design of Microservice Architectures Software Engineering and Formal Methods*. University of Applied Sciences and Arts Dortmund, Institute for Digital Transformation of Application and Living Domains, Dortmund, Germany.
- Radulović, A., Sladić, D.A., 2017. Towards 3D cadastre in Serbia: development of Serbian cadastral domain model. *Int. J. Geo Inf.* 6 (6), 312.
- Stopford, B., 2018. *Designing Event-Driven Systems - Concepts and Patterns for Streaming Services with Apache Kafka*. O'Reilly Media, Inc, Sebastopol, CA.
- Swiss Confederation, 2007. *Federal Act on Geoinformation*. (<http://www.admin.ch/op/c/en/classified-compilation/20050726/index.html>). (Accessed 30 September 2020).
- Van Oosterom, P., Lemmen, C., Uitermark, H., 2013. *ISO 19152:2012, Land Administration Domain Model published by ISO*. Abuja, Nigeria, 6–10 May 2013: FIG Working Week 2013 Environment for Sustainability.
- Vernon, V., 2013. *Implementing Domain-Driven Design*. Addison-Wesley Professional.
- Vos, J., Roes, B., 2020. *Introducing a Complete New Land Register & E-conveyancing System - A description of the complete renewal of the Dutch Land Register & e-conveyancing system*. Washington: World Bank Land and Poverty Conference 2020. (https://www.conftool.com/landandpoverty2020/index.php/11-07-Vos-571_paper.pdf?page=downloadPaper&filename=11-07-Vos-571_paper.pdf&form_id=571&form_version=final). (Accessed 11 October 2020).
- Vučić, N., Markovinović, D., Mičević, B., 2013. *LADM in the Republic of Croatia – Making and Testing Country Profile*. In: 5th Land Administration Domain Model Workshop, 24–25 September 2013, Kuala Lumpur, Malaysia.
- Yin, R.K., 2014. *Case Study Research, Design and Methods, fifth edition*. Sage Publications, Inc, Thousand Oaks, CA.
- Zaragoza, M.G., Kim, H.-K., Yeo, H., Hwang, H.J., Ramos, C., Goreti, M., 2016. *Structuring CAMA (Context Area Mobile Applications) in SOA (Service Oriented Architecture) and MDA (Modern Driven Architecture)*. *Adv. Sci. Technol. Lett.* 139 (ASEA 2016), 241245.
- Zevenbergen, J., 2002. *Systems of Land Registration - Aspects and Effects (Ph.D. thesis)*. Delft University of Technology, Netherlands Geodetic Commission (NGC), Delft, Netherlands.
- Zevenbergen, J., Frank, A., Stubkjær, E., 2007. *Real Property Transactions - Procedures, Transaction Costs and Models*. IOS Press BV, Netherlands.